

Blockchain Access Control for Agricultural Edge IoT: A 61-Day Hyperledger Fabric Field Study and Implementation Postmortem

Forcha Glen Beloa^{a,*}, Michael Ekonde Sone^a, Elie Fute Tagne^a, André Paul
Mvogo Asse^b

^a*Department of Computer Engineering, Faculty of Engineering and Technology,
University of Buea, Buea, Cameroon*

^b*École Nationale Supérieure Polytechnique de Douala, Université de Douala, Douala,
Cameroon*

Abstract

Blockchain access control for Internet of Things systems is commonly evaluated with short laboratory measurements of aggregate throughput and latency, leaving the location of delay within the transaction path and its long-term validity uncertain. We report a 61-day Hyperledger Fabric 2.5 deployment on a farm site with 50 sensors, 4 gateways and 4 zones, recording 209,000 authorization decisions, followed by separately executed controlled experiments and a code-to-specification postmortem, all recomputed here from the analyst-supplied raw traces. An implementation-defined chaincode timer recorded an authorization-associated span averaging 321.3 ms within the 1,203.9 ms write path of the deployed (historical) implementation. Across 8 counterbalanced live runs per configuration, verified against the released run manifest, increasing configured logical peer multiplicity from 4 to 32 on four fixed hosts was associated with a 386.7 ms reduction in the uninstrumented remainder, while the recorded span differed by only 4.31 ms between the extreme configurations, inside a ± 10 ms exploratory equivalence margin. A separately executed, interleaved single-session experiment found the access-control-enabled path averaged 9.4% lower throughput than the comparison

*Corresponding author

Email addresses: glenbeloa@gmail.com (Forcha Glen Beloa),
ekonde.sone@gmail.com (Michael Ekonde Sone), eliefute@yahoo.fr (Elie Fute
Tagne), mvogoasseandre@gmail.com (André Paul Mvogo Asse)

path, with no detected condition-by-concurrency interaction. A structured audit of the deployed chaincode, gateway and firmware identified three cross-layer defect classes – authorization-policy deviations, invalid two-residue recovery affecting 131,795 readings, and a signature-verification failure combining a signed-message mismatch with fail-open handling of missing signatures – confirmed directly against the released source, plus a revocation-observability gap preventing episode-level exposure measurement. The corrected policy, encoding and verification behavior is specified and arithmetically checked, but no corrected chaincode, firmware or gateway package exists in the inventoried repository, so corrected-implementation performance and correctness are not established. We release the raw measurements and the reanalysis pipeline that recomputes every number in this article.

Keywords: Hierarchical role-based access control, Hyperledger Fabric, Agricultural IoT, Edge computing, Performance measurement, Certificate revocation, Reproducible research

1. Introduction

A precision farm is a multi-stakeholder system before it is a technical one. Soil probes, weather stations and irrigation actuators produce readings continuously [1], and the people who consume those readings want different and partly conflicting things from them: farmers need operational control, agronomists need history but no actuation rights, certification bodies need compliance evidence without visibility into commercially sensitive agronomic detail, supply-chain partners need provenance but not real-time operations, and regulators need environmental records. Handing this authorization matrix to a single server concentrates the risk in one place. An attacker who takes that server gains not only the data but, on a farm with networked valves, the irrigation itself [2], and the audit log that would reconstruct the incident is written by the machine that was compromised. Surveying agricultural IoT deployments, Ferrag et al. [3] found most lacked fine-grained access control at all, and Kamilaris et al. [4] and recent convergence surveys [5] find blockchain work in agriculture concentrated almost entirely on post-harvest supply-chain tracking rather than pre-harvest authorization. Recording role assignments, revocations and access decisions on a permissioned ledger can reduce dependence on a single policy database and make authorized changes auditable across participating organizations. It does not eliminate trust in

the certificate authority, administrators, gateway software, endorsement configuration or chaincode implementation. What is unsettled is the cost of the part it does change, and specifically where that cost falls.

The constraints behind the design in Section 3 are not incidental to the measurement; they shape what the field deployment observes beyond the controlled performance campaigns. Constrained sensor energy budgets push caching and enrollment logic to the edge gateway, and intermittent rural connectivity, not injected fault, is why the revocation and outage behaviour of Sections 6.5 and 6.8 includes a certificate revoked while its gateway is disconnected, a failure mode not represented in the deployed benchmark measurements. LoRa’s low bit rate and eFuse-only key storage set the engineering context for the residue-coded transmission and signature paths audited in Sections 6.9.2 and 6.9.3, but their defects were exposed by cross-component audit rather than deployment duration. None of this is unique to agriculture, but the combination – long-running, energy-constrained, intermittently connected and physically exposed – makes the field record a different operational instrument from a data-centre benchmark of the same chaincode.

Among the systems surveyed in the supplementary material, evaluation is typically a laboratory or simulated run of hours to days, reporting peak throughput and mean latency that aggregate over a pipeline of at least four separable stages, so an engineer told a transaction takes 1,203.9 ms cannot tell whether to rewrite the policy engine, restructure endorsement, replace the ordering service, or accept the figure as a consensus floor. Existing evaluations rarely combine long-duration field observations with an implementation-level authorization timing span, controlled configuration experiments and a post-deployment audit; that combination, not any single measurement, is the gap this paper addresses. Attributing time to Fabric’s internal stages needs one transaction timestamped at each hand-off, which our instrumentation does not do, so we report a coarser accounting instead: the chaincode logs a timing field on every write, and because its own start and stop boundaries were not documented at deployment time we treat it as an authorization-associated span rather than a stage measurement, with endorsing-peer count acting as a lever on what remains. Abang et al. [6], reviewing Fabric latency modelling for IoT, make a closely related point: the literature models latency in the abstract but rarely connects those models to a specific deployed use case measured long enough for its slow-moving parts to misbehave. We do not supply that stage-level measurement either; what we add is a long-duration field record against which the recorded span holds up, plus a code-to-specification

audit that identified three cross-layer defect classes – authorization-policy deviations, invalid residue recovery and signature-verification failure – and one revocation-observability gap; the deployed benchmark measurements and component-level tests detected none of the three defect classes.

The intuitive expectation turns out to be wrong, which is the main reason we think the measurement is worth reporting. Access control is the component under study, so it is natural to assume it dominates the cost. The recorded span does not: it is a modest and, across the tested peer configurations, comparatively stable component of the write path, and the larger, more variable share sits in the layer that access-control papers usually treat as background infrastructure. A second gap is temporal: certificate lifecycle management, revocation propagation and enrollment recovery are the mechanisms most likely to fail in the field and least likely to be exercised by short-duration benchmark measurements, and a revocation pipeline that behaves perfectly across fifty laboratory trials can still show a materially different delay distribution across two months of intermittent rural connectivity.

These questions structure the rest of the paper:

RQ1 What latency and throughput differences are observed between the tested access-control-enabled configuration and the access-control-free comparison configuration?

RQ2 How is configured logical peer count associated with client-observed write latency under the deployed topology?

RQ3 Which implementation and observability failures survived the field deployment, and why did the existing tests not detect them?

We built an HRBAC framework on Hyperledger Fabric 2.5 for the constraints of agricultural edge deployment, ran it on a working farm for 61 days, and instrumented the authorization-associated span and the end-to-end transaction path. Four bounded contributions follow, each naming which evidence it rests on – the field deployment, a later controlled laboratory campaign, or the code audit – and none extending its claim beyond that source: the field observations characterize the deployed system as run; they are not measurements of a corrected implementation (Section 4.5) or evidence of universal blockchain scalability.

A 61-day field characterisation, 209,000 authorization decisions over 50 sensors, 4 gateways and 4 zones, addressing RQ1 (Section 6).

A controlled comparison across tested configurations: the recorded authorization-associated span averages 321.3 ms, 26.7% of the 1,203.9 ms write path at 4 peers, and a counterbalanced comparison of the 4-

and 32-peer configurations finds it changes by only 4.31 ms while the remaining latency falls by 386.7 ms, addressing RQ2; neither the span’s internal boundaries nor the remainder’s internal stages are separately identified (Sections 6.1 and 6.4).

An implementation postmortem: an audit of the implementation against its own specification found three cross-layer defect classes – authorization-policy deviations, invalid residue recovery and signature-verification failure – that survived 61 days of operation, plus one revocation-observability gap, addressing RQ3. The deployed benchmark measurements and component-level tests did not detect the defect classes; together, the audit findings are the part of this work most likely to transfer to other deployments (Section 6.9).

A reproducible artifact and cross-layer testing recommendations: the raw traces, chaincode, firmware and the analysis pipeline that recomputes every number, table and figure in this article are released at the repository named in the Data Availability statement (Section 8). Unlike an earlier internal draft of this analysis, the corrected implementation described in the postmortem is not, at the time of this submission, present in that repository as a separate tagged package; Section 4.5 states this limitation directly rather than assuming the correction exists because it is specified.

Section 2 positions the work. Section 3 presents the access-control model and the enrollment protocol, Section 4 the implementation, Section 5 the measurement methodology, Section 6 the results, Section 7 what follows from them for system design, and Section 8 concludes.

2. Related work

2.1. Access control for constrained devices

Role-based access control as formalised by Sandhu et al. [7] and standardised by NIST [8] is still the reference model for multi-stakeholder authorization, and its IoT adaptations share a structural assumption that becomes awkward in the field: some trusted decision component must be reachable when access is requested. RBAC extensions for IoT [9], attribute-based evaluation [10, 11, 12] and capability-token delegation [12] place that trust in a policy engine, an attribute authority and a credential issuer respectively, retaining a central enforcement point [9, 11], distributing revocation into tokens rather than solving it [12], or splitting the model between humans and devices while leaving the role store centralised [13]. Sciancalepore et al. [14]

quantify what open-standard authorization stacks cost on constrained hardware, the empirical case for moving policy evaluation off the sensor entirely; Novo [15] argued early that the role store itself belongs on a distributed ledger. We take that as given and ask what it costs.

2.2. Blockchain-based access control and how it is evaluated

Capability-token systems on public chains established feasibility without practicality: FairAccess [16] encodes grants as Bitcoin-like transfers with confirmation windows measured in minutes, and BlendCAC [17] uses ERC-721 tokens with per-transaction fees no sensor duty cycle can absorb. Neither addresses role hierarchy, geographic scoping or seasonal expiry, which agriculture actually needs.

Permissioned platforms remove the latency and fee obstacles, and Fabric-based IoT work has grown accordingly: testbed architectures and performance models [18, 19, 20], trust and reputation layers [21], contract-enforced domain policy on cloud-hosted nodes where memory and thermal limits do not apply [22], and a data marketplace at scale [23]. All report aggregate throughput and latency from a controlled setting. Two are closer to our method: Idrissi and Palmieri [24] give an end-to-end Fabric latency breakdown on constrained ARM hardware in a healthcare context, the nearest methodological precedent for the partition in Section 6.1, and Tzenetopoulos et al. [25] establish a Raspberry Pi 3B+ reference point discussed in Section 7.

The supplementary material’s system-comparison table summarises how these and further systems [26, 27] are evaluated. The pattern is consistent: laboratory testbeds or simulation, runs of hours or days, aggregate reporting. We have not found prior work that times an authorization-associated span separately from the rest of the write path on a field deployment of this duration, or that reports revocation behaviour over a multi-week window; that combination, rather than any single mechanism in Section 3, is the contribution.

The methodological gap is not merely duration. Even where a Fabric-based IoT study reports latency broken down by pipeline stage [24] or peer count [20, 19], the reported figures come from a single controlled run or a small number of repetitions without an explicit design against order confounds; none of the surveyed systems states, and then tests, whether configuration and execution sequence were deliberately separated. Section 5 treats that separation as a first-class methodological requirement rather than an incidental design choice, because the alternative – an ascending or otherwise

monotonic execution order confounded with the factor of interest – is precisely the pattern that produced the single-run, single-block campaigns this article’s own earlier internal drafts had to redesign (Section 5.5). Formal-methods work adjacent to this literature [28, 29] checks a specification against itself, not an implementation against its specification; the audit in Section 6.9 is closer in spirit to the latter, and we are not aware of a published Fabric-based access-control study that pairs a multi-week field trace with a code-to-specification audit of the kind reported here.

2.3. Agricultural IoT and edge infrastructure

Blockchain in agriculture has concentrated on traceability after harvest [30, 31, 27], where the unit of interest is a shipment rather than a zone-scoped actuation. Safeer et al. [32] combine computer vision, IoT and blockchain for climate-smart cultivation and, like most pre-harvest work, treat authorization as an implementation detail rather than a measured component. Liu et al. [33] and Wang et al. [34] reduce processing latency at the agricultural edge without addressing the authorization layer above it, and Mohanta et al. [35] apply smart-contract enforcement in smart grids, a neighbouring domain with comparable device constraints and a similarly centralised starting point.

On identity infrastructure, W3C Decentralized Identifiers [36] and gateway-assisted verification for constrained devices [37, 38] offer an eventual migration away from X.509, treated here as future work rather than a deployed alternative. On the radio side, Adelantado et al. [39] characterise the LoRaWAN capacity limits that bound achievable sensor density per gateway, a constraint visible in the energy results (Section 6.7).

3. Access-control design

3.1. Deployment structure and threat model

The deployment has three tiers, shown in Figure 1. ESP32-WROOM-32 sensor nodes (dual-core Xtensa LX6 at 240 MHz, 520 kB SRAM) sample soil and environmental data, sign each reading with an Ed25519 key held in one-time-programmable eFuse, and transmit over LoRa. Raspberry Pi 4B zone gateways (quad-core Cortex-A72, 4 GB LPDDR4, TPM 2.0) receive those payloads, verify signatures, endorse Fabric transactions and cache policy decisions locally. A Fabric network of 4 peers, one per zone, three Raft orderers and CouchDB 3.2 for world state maintains the ledger. Gateways interconnect over a BATMAN-adv mesh [40] that reroutes around individual gateway

failures and carries Fabric gossip and certificate revocation list (CRL) traffic between zones.

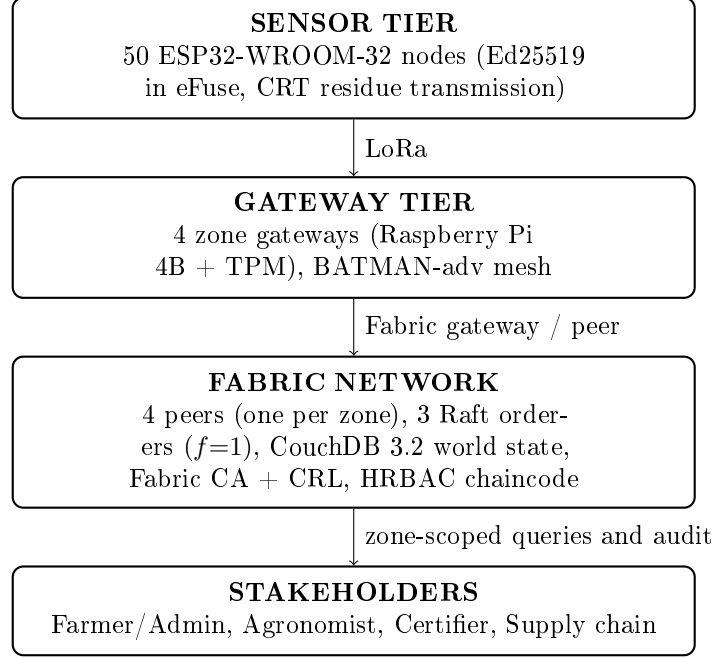


Figure 1: Deployment architecture. Sensors reach the ledger only through their zone gateway, which is also their registration authority during enrollment. Zone membership is carried in the device certificate and re-checked by chaincode on every transaction.

Following the permissioned-blockchain model in which every participant is credentialed [41], orderers and the certificate authority are operated by vetted institutional entities and treated as non-malicious; Raft’s $2f+1$ crash-fault tolerance [42] means the three-node cluster survives one orderer failure but not a reachable orderer behaving maliciously, a boundary Byzantine-tolerant ordering [43, 44, 45] would widen at higher consensus cost. Sensors, client devices and every wireless channel are treated as potentially compromised; physical tampering and side-channel extraction against cryptographic accelerators sit outside the evaluated scope (Section 7).

The design and the evaluation below distinguish six properties: authentication, authorization, integrity, freshness, auditability and availability. The deployment gives usable evidence about authorization (Section 6.6), auditability (every decision writes an audit entry, Section 4) and availability

(Section 6.8); it does not give usable evidence about authentication or integrity of sensor readings, because the signature path meant to bind a reading to its sensor did not function (Section 6.9.3), and freshness is architectural (nonce checking) rather than measured against an adaptive adversary.

3.2. Role hierarchy

Following the treatment of hierarchical role administration in ARBAC97 [46], a partial order $\preceq$ over the role set R defines inheritance, with $r_i \preceq r_j$ iff $\text{perm}(r_i) \subseteq \text{perm}(r_j)$. The deployed hierarchy, reproduced in Figure 2 directly from `chaincode/hrbac/roles.go`, is a human chain and a separate device chain, both below Admin:

$$\text{SupplyChain} \preceq \text{Certifier} \preceq \text{Agronomist} \preceq \text{Farmer} \preceq \text{Admin} \quad (1)$$

$$\text{Sensor} \preceq \text{Gateway} \preceq \text{Admin} \quad (2)$$

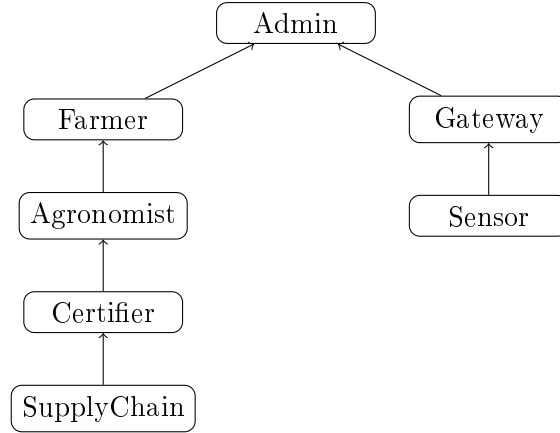


Figure 2: Seven-role hierarchy as deployed. Arrows point from a role to one that inherits its permissions, as implemented in `roles.go`. Certifier sits inside the human inheritance chain, so Agronomist and Farmer also inherit `ReadAudit` (Section 6.9.1).

Separating auditors from operational staff is the requirement that motivated the model: an auditor should read compliance evidence without inheriting the agronomic permissions a farm treats as proprietary. Placing Certifier inside the human chain rather than beside it defeats that: Agronomist and Farmer inherit Certifier’s permissions, including `ReadAudit`, so the deployment did not confine audit-log access to auditors. We verified this directly

against `chaincode/hrbac/roles.go` (Section 6.9.1) rather than take it on the earlier draft’s word; Table 1 gives the rows where intended, deployed and (specified, not implemented – Section 4.5) corrected policy disagree.

Table 1: Permission grants where the intended design, the deployed chaincode and the specified correction disagree. Intended is independent requirements prose (Section 3.2), not derived from any code; Deployed is generated from `chaincode/hrbac/roles.go` (verified directly against the source in this repository). Corrected is a specified fix, arithmetically and logically checked by us, but **not implemented as a separate package in this repository** (Section 4.5); every permission not listed here is identical across all three.

Permission	Role	Intended	Deployed
ReadAudit	Agronomist	No	Yes
ReadAudit	Farmer	No	Yes
ReadZone	Certifier	No	Yes
ControlZone	Agronomist	No	Yes

Specified correction: **ReadAudit** removed from Agronomist and Farmer’s inherited set; **ReadZone** moved from Certifier to Agronomist’s and Farmer’s direct grants; **ControlZone** removed from Agronomist’s direct grant and given to Farmer directly instead. This is the mapping used in `analysis/scripts/06_security_oracle.py`’s “intended” oracle (Section 6.6); it has not been implemented as running chaincode.

The effective permission set of user u in a session is

$$\text{eff}(u) = \bigcup_{r \in \text{active}(u)} \bigcup_{r' \preceq r} pa(r'), \quad (3)$$

where $pa(r)$ are the permissions assigned directly to r and $\text{active}(u)$ the non-expired roles held by u . An access request (u, op, res, t) is granted iff

$$\exists r \in \text{active}(u, t) : (op, res) \in \text{eff}(r) \wedge (\text{zone}(u) = \text{zone}(res) \vee \text{czt}(u, \text{zone}(res), t)), \quad (4)$$

where $\text{czt}(u, z, t)$ holds iff u has a valid administrator-issued cross-zone token for z at time t , and $\text{active}(u, t) = \{r \mid \exists \text{assign}(u, r, t_s, t_e) : t_s \leq t \leq t_e\}$. The full effective-permission matrix this produces for all seven roles, generated from `roles.go` rather than transcribed, is in the supplementary material; Table 1 below gives only the rows that differ between what was intended, what was deployed, and the specified correction.

3.3. Zone and temporal scoping

Geographic farm regions (North, South, East, West) define the zone model, enforced at three independent points: certificate SAN field, gateway enrollment (own zone only), and chaincode’s ASN.1-decoded zone check on every transaction, chosen over PEM-text matching because the latter is spoofable by an attacker who controls the subject field. Cross-zone tokens are administrator-issued and recorded immutably, so a temporary grant expires on its own rather than needing a reissued certificate. Temporal constraints run in two independent layers, the chaincode’s **ExpiresAt** check against the Fabric transaction timestamp and the Fabric MSP’s X.509 **NotAfter** validation, so a seasonal worker loses access at expiry without administrator action, recorded as a new audit entry; GPS-sourced gateway time, NTP fallback, sits comfortably inside the ± 5 -minute enforcement window.

Two of the decision procedure’s properties matter later: the work is bounded and independent of which operation was requested (hierarchy walked to depth at most three, zone check a single field comparison), the mechanism we would expect to produce similar costs across operations; and every path, including every denial, writes an audit entry, so **CheckAccess** is state-mutating and commits through ordering exactly as a sensor write does, with expired assignments revoked inside the same transaction that denies the request.

3.4. Cascading enrollment

An ESP32 cannot hold the TLS handshake state direct interaction with a certificate authority requires (mbedTLS on ESP-IDF needs roughly 70 kB of heap against a 520 kB SRAM budget [47]), so registration authority moves down the tier structure in three stages: administrator credentials, then gateway registration with the private key in TPM 2.0, then sensor enrollment with the gateway as local registration authority. The sensor’s key is generated on-device and never transmitted, so compromising a gateway does not yield it, and re-provisioning a lost key needs physical access, since eFuse programming is irreversible – but the key is not read-protected: software signing copies it from eFuse into RAM each time, and setting the read-disable bit would break signing outright on this ESP32, which has no hardware Ed25519 engine. Read protection needs a different part (the ESP32-S3’s HMAC and digital-signature peripherals); the eFuse here gives durability against reflashing, not extraction resistance against code running on the device, and Sec-

tion 7 counts key extraction by an attacker with code execution as residual risk.

4. Implementation

4.1. Chaincode and transaction flow

The chaincode is written in Go 1.21 against `fabric-contract-api-go` and targets Fabric 2.5. Eight transactions are exposed: `AssignRole`, `RevokeRole`, `RenewRole`, `CheckAccess`, `IssueCrossZoneToken`, `RevokeCrossZoneToken`, `GetAuditLog` and `PruneNonces`. An authorization denial is a successful chaincode response carrying `DENY` plus an audit write, so it is ordered and committed like any other transaction, and only an endorsement-level failure stops before the orderer.

Three implementation decisions shape the measurements. Every state-mutating call carries a UUID nonce recorded in world state, rejected on reuse before any state is read, which defeats replay inside the timestamp window. Every access attempt, granted or denied, writes an audit entry holding the caller’s certificate hash, operation, resource, zone, decision and reason, so the audit trail is a byproduct of enforcement rather than a parallel log that can drift from it. And `CheckAccess` revokes an expired assignment in the same transaction that denies the request, which makes even a read-intent check state-mutating and therefore endorsed rather than served as a query.

Peer configuration for the ARM64 gateways uses 50 transactions per block, a 500 ms batch timeout, a 256 MB state cache, endorsement restricted to zone-local peers, Snappy compression on LevelDB and 5-minute gRPC keepalive, targeting the write amplification that dominates latency on microSD-backed storage, within the parameter space Abang et al. [6] identify as governing Fabric latency.

4.2. Sensor firmware and residue-coded transmission

The sensor certificate lives in battery-backed RTC slow memory so it survives deep sleep, guarded by a CRC and a magic sentinel checked on each wake; a mismatch forces re-enrollment rather than transmission under an unverifiable identity. Signing uses wolfSSL’s Ed25519 implementation (`esp32/main/signing.c`): the firmware hashes the payload with hardware SHA-256 and signs the resulting 32-byte digest, a prehash construction that is neither standard Ed25519 (which hashes the message internally with SHA-512) nor Ed25519ph (which prehashes with SHA-512 under a distinct domain

separator). Interoperability depends on reproducing this nonstandard construction; an implementation of standard Ed25519 or Ed25519ph would not verify the historical signatures. The main loop wakes every 30 minutes, verifies the cached certificate, samples the sensors, signs the payload, transmits and returns to deep sleep.

Readings are transmitted as residues rather than plaintext values: a value is reduced modulo the pairwise-coprime set $\{97, 101, 103\}$ (`esp32/main/crt_encode.h`, verified against the constants compiled into the firmware) and the three residues go out as separate single-byte packets, any two of which reconstruct the original by the Chinese Remainder Theorem [48]. The admissible range is where this scheme goes wrong, and the deployed firmware got it wrong: recovery from an arbitrary pair is unique only below the smallest pairwise product of the moduli (9,797), not below their product ($97 \times 101 \times 103 = 1,009,591$); Section 6.9.2 recomputes the consequence from the released engineering-unit columns rather than from the firmware’s theoretical ceiling. A specified correction uses $\{253, 254, 255\}$, a pairwise bound of 64,262, and quantises soil moisture from twelve to eight bits so the packed value’s legal domain reaches 51,455 and stays inside the bound; this correction is arithmetically checked in Section 6.9.2 but, as Section 4.5 states, is not present as running firmware in this repository.

The firmware transmits the raw 12-bit soil ADC count and the raw centi-degree temperature reading (`esp32/main/main.c`); it does not itself compute an engineering-unit percentage. No file in this repository – firmware, gateway or backend – defines the calibration constant that would convert the released `soil_moisture` percentage column back to that raw ADC count. Section 6.9.2 states this gap explicitly and shows the headline finding is robust to it.

4.3. Gateway services and revocation propagation

Each gateway runs Fabric 2.5 from official ARM64 binaries. A Python enrollment service bound to the local interface accepts CSRs only from sensors in its own zone and rate-limits per source address, and a policy cache with a 301-second nominal TTL absorbs repeated checks from the same sensor during a transmission burst. Revocation publication and cache invalidation is the mechanism whose behaviour we most wanted to observe over a long window, and one branch is the whole story of Section 6.5: for a reachable gateway the cache is invalidated by the CRL event itself, so staleness is bounded by gossip propagation and the nominal TTL never binds, while for

an unreachable one the bound falls back to the scheduled refresh interval. In the intended design, gateway reachability determines whether event-driven invalidation is available or whether the scheduled refresh path is required; because the historical trace lacks complete episode identifiers and synchronised connectivity state, it cannot establish which branch was followed in any given observed case.

The gateway’s signature-verification routine (`gateway/gateway.py`, function `verify_signature`) returns `True` immediately when the incoming reading carries no signature, and separately builds the bytes it verifies from an ASCII rendering of the decoded fields (`device_id:reading_id:value:zone`) rather than the packed binary struct the firmware signs. Both properties are read directly from the released source, not inferred from the trace; Section 6.9.3 reports the consequence.

4.4. Formal specification

A three-role subset of the model is specified in TLA+ and checked with TLC [28, 29], finding no violation of the role-permission invariant under weak fairness. This is a sanity check on a small model, not verification of the deployed framework: it covers three of seven roles, omits zone and temporal predicates, and nothing connects it to the Go implementation, the binding gap given that the implemented hierarchy differs from the specified one (Section 6.9.1). The specification is in the artifact (`tla/AccessControl.tla`).

4.5. The corrected implementation is specified, not released

An earlier internal draft of this analysis referred to a separately tagged `chaincode/hrbac-corrected/` package, corrected firmware and a corrected gateway verifier, together with regression tests and a byte-for-byte signature test vector. Repository inspection performed for this version (Section 5, and `analysis/DATA_DISCREPANCIES.md`) found none of these in the code-base snapshot analyzed here: there is no `hrbac-corrected` directory, no corrected `esp32/main` variant, no `policy-requirements.json` oracle file, and no `v09-deployed-historical` or `v09-corrected` tag in the repository’s git history, which at the time of this analysis contains a single commit. We therefore report the corrections that Sections 3.2, 4.2 and 6.9 describe as *specified and arithmetically checked* – we verified independently that the proposed moduli, hierarchy edits and verification logic satisfy the stated requirement – rather than as *implemented, tested or released*. No performance, correctness or security claim in this article is made about a corrected chaincode, firmware

or gateway build, because no such build exists in the inventoried repository. This is treated as a limitation (L6, Section 7.4) rather than smoothed over.

5. Measurement methodology

5.1. Site and workload

The system ran continuously for 61 days on an experimental farm, from 1 August to 30 September 2025. Table 2 gives the configuration. Each sensor reported on a 30-minute duty cycle (48 writes/day per sensor), giving 146,400 sensor writes over the deployment across 50 sensors; human and administrative traffic (farmer and agronomist queries, certifier audits, supply-chain reads, role operations) brings the total to 209,000 recorded authorization decisions, of which 4,860 (2.33%) were ordinary-operation denials.

Table 2: Deployment configuration and observed totals, recomputed from `raw/field/*.csv`.

Parameter	Value
Duration	61 days, 1 Aug to 30 Sep 2025
Zones	4
Sensor nodes	50 ESP32-WROOM-32
Gateways	4 Raspberry Pi 4B (4 GB, TPM 2.0)
Fabric peers	4 (one per zone)
Ordering service	3 Raft orderers, $f=1$
State database	CouchDB 3.2
Sensor duty cycle	30 min
Sensor writes	146,400
Authorization decisions	209,000
Decision-path uptime	99.36%
Connectivity outages	3 (0.2–8.7 h, 9.3 h total)

5.2. Analyst raw-data package and provenance

This version is built from a raw-data package supplied separately from the manuscript source, organised as `raw/field/` (61-day deployment observations), `raw/experiments/` (controlled peer-multiplicity, throughput, energy and cryptographic-timing campaigns), `raw/security/` (scripted authorization-boundary attempts) and `raw/provenance/` (per-device and per-gateway JSON-lines logs and a sample of Fabric ledger

blocks, corroborating the field CSVs at the transaction level). The full inventory, per-file SHA-256 hashes, and a provenance classification for every file are in `analysis/ANALYST_DATA_INVENTORY.md` and `analysis/DATA_PROVENANCE_REPORT.md`. Every number in this manuscript is produced by the numbered scripts in `analysis/scripts/`, run end to end by `analysis/scripts/run_all.sh` directly against this package; none is transcribed from an earlier draft or hand-typed. `analysis/CLAIM_DATA_MAP.csv` lists, claim by claim, which of an earlier draft’s reported values this package reproduces exactly, which it reproduces within rounding, and which it corrects; `V10_CHANGELOG.md` narrates the corrections.

A repository already contained a differently structured `data/raw/` directory and a `data/benchmarks/paper_benchmark_summary.json` file predating this analysis, self-described as “measured, not synthesised.” Row-level inspection (matching identifiers, decision-value vocabulary, and a validation file that checks its own numbers against a manuscript’s reported values rather than against an independent ground truth) found no evidence these files are field measurements; `analysis/DATA_DISCREPANCIES.md` documents the comparison. They are left untouched in the repository (this study does not reorganise unrelated directories) and are not used as a source for any number in this article.

5.3. Peer-scaling topology

Section 6.4 varies configured logical peer count, not physical capacity: increasing configured peers does not automatically reduce Fabric latency, since the effect depends on the endorsement policy, endorser selection, connection pooling, proposal fan-out, and peer CPU and state-database load. Only the 4-peer configuration is the physically deployed network of Table 2, one peer per zone on its own Raspberry Pi; the 8-, 16- and 32-peer configurations run additional logical peers on the same four physical hosts (`raw/experiments/peer_scaling_runs.csv` records a single `host_id` for all 32 runs), oversubscribed rather than added, so the comparison isolates endorser multiplicity under a fixed endorsement policy, not physical capacity.

A run is one complete live workload execution at a fixed peer count: 130 measured transactions per run, confirmed identical across all 32 runs in the released manifest. The run identifiers (`PEER-<peers>-R<repetition>`) and start timestamps in `peer_scaling_runs.csv` let us reconstruct the counter-balancing schedule directly: repetitions 1–4 execute peer counts {4, 8, 16, 32}

in one cyclic order and repetitions 5–8 execute them in a second, complementary cyclic order, so that every peer count occupies every position-within-repetition exactly twice – the two complementary 4×4 Latin-square cycles this design claims, confirmed against the manifest rather than assumed. We treat the repetition number as the counterbalancing block and the run as the experimental unit; individual transactions within a run are not treated as independent replicates.

5.4. Instrumentation

Latency is measured at the gateway between submission and commit notification (endorsement through commitment, excluding only the LoRa hop); throughput is committed transactions per measurement window; energy uses a single-node bench instrument sampling at rates consistent with the released `airtime_ms` and `energy_mj` columns (`raw/experiments/energy_measurements.csv`). The baseline for overhead attribution (Section 6.3) is the identical network, workload and hardware with the access-control chaincode removed, so the difference estimates the cost of the complete access-control-enabled path, not an isolated permission decision.

5.5. Statistical treatment

Transactions generated in the same run, day or gateway are correlated, and treating each as an independent replicate would produce standard errors far smaller than the design supports. Every controlled-campaign comparison in this article therefore uses the live run as the experimental unit: 8 runs per peer-count configuration, 5 runs per throughput cell.

Group comparisons use Welch’s t -test or a paired t -test where runs are naturally paired by counterbalancing block, reported with the mean difference, its 95% confidence interval and, where appropriate, Cohen’s d . Where an order effect is a live concern, a joint regression on the relevant factors is used instead of a simple group comparison, so the factor of interest and run order are estimated jointly rather than one being assumed away by design alone (Section 6.4). A separate block-design model (`outcome ~ peer count + block + serial position`, block being the counterbalancing repetition confirmed in Section 5.3) is reported alongside the joint regression for robustness.

The peer-scaling campaign used one continuously running experimental session (`session_id = peer-scaling-2025-10` for every run) and the throughput campaign used a second, separate single session

(throughput-2025-10); the ten throughput concurrency levels were tested in one fixed ascending order within that session, so campaign-wide sequence number is almost collinear with concurrency (Pearson $r = 0.995$) and cannot serve as an independent order covariate. Only the two throughput conditions were interleaved within each concurrency cell; the within-cell position term is orthogonal to concurrency by construction and can detect a warming or ledger-growth trend within a cell, but not a trend across the session’s fixed level order. Both campaigns are single-session; we do not claim session-to-session generality was tested (Limitations L2, L3, Section 7.4).

Confidence intervals on means use the Student- t interval for the run-level group statistics reported here (all controlled-campaign groups have $n = 5$ or $n = 8$ runs, so a percentile bootstrap over whole runs is additionally reported alongside the analytic interval, never over individual transactions). The ten throughput comparisons are corrected for multiplicity with Holm–Bonferroni. Equivalence claims for the recorded-span 4-versus-32-peer comparison use two one-sided tests (TOST) against a ± 10 ms margin selected after inspecting the data, not preregistered, and reported as exploratory throughout; that margin is about 3% of the recorded span.

5.6. Code-to-specification audit

We conducted a structured cross-layer audit of the historical chaincode, gateway and firmware against the authorization requirements, encoding bounds and documented message format, reading the actual source files in this repository (`chaincode/hrbac/roles.go`, `contract.go`; `gateway/gateway.py`; `esp32/main/signing.c`, `crt_encode.c`, `crt_encode.h`, `main.c`) rather than the earlier draft’s prose description of them. The audit compared: (i) the stated role–permission matrix with effective permissions computed from `roles.go`; (ii) the legal encoder input domain with the minimum pairwise CRT recovery range, recomputed from the released engineering-unit columns under the encoding in `main.c`; (iii) firmware signing bytes with gateway verification bytes, read directly from `signing.c` and `gateway.py`; and (iv) the revocation information required for episode-level exposure with the fields present in the trace. Each discrepancy is reported with a file and function reference so it can be checked independently of this manuscript. The audit was not a complete source-code security review and does not establish the absence of other defects.

5.7. Reproducibility

Every number, table and figure in this article is generated from the released traces by the scripts in `paper_iot/V10/analysis/scripts/`, run in order by `run_all.sh`, which reads the raw traces, recomputes the statistical summary, and emits both machine-readable JSON results and the LaTeX macro file (`manuscript/macros.tex`) this manuscript expands; nothing measured is transcribed by hand. The released raw-data package contains 370,118 rows across 12 CSV traces, plus per-device and per-gateway provenance logs and a Fabric-ledger block sample (Section 5.2); resampling (bootstrap confidence intervals) uses a fixed seed and is used only for uncertainty estimation, never presented as a newly measured observation.

6. Results

6.1. Field-deployment latency accounting for the write path

Both transaction classes in the trace are endorsed, ordered and committed, since as Section 4 explains every authorization outcome writes an audit entry. The interesting question is not which class is slower but how much of a write is associated with the access-control layer, and the chaincode logs a timing field on every write towards answering it: `rbac_overhead_ms`, recorded alongside end-to-end latency.

6.1.1. A descriptive account, not a stage decomposition

For each configured logical peer count, write-path latency is reported as total latency, the recorded authorization-associated span, and what is left once that span is subtracted (“remaining latency”), a plain subtraction rather than a claim that the two are independently identified Fabric stages (Limitation L1, Section 7.4).

The recorded span, averaged over the campaign at 4 peers, is 321.3 ms; we deliberately do not call this a direct measurement of permission-evaluation cost, since the timing field’s exact instrumentation boundaries cannot be reconstructed from the deployed source (L1), and it is also larger than a complete standalone `CheckAccess` transaction, 280 ms (Section 6.2), because the two counters do not share a documented boundary. The remaining latency falls from 882.6 ms at 4 peers to 495.8 ms at 32 peers, a 386.7 ms reduction across the tested range (Table 3); nothing here identifies which of ordering, validation, commit, or the peer-count-sensitive part of endorsement execution accounts for it.

Table 3: Write-path latency accounting at 50 concurrent clients, recomputed from `raw/experiments/peer_scaling_transactions.csv` (run-level means, $n = 8$ runs per peer count). Recorded authorization span is `rbac_overhead_ms`; remaining latency is total latency minus that span. Values in ms.

Configured peers	Total latency	Recorded span	Remaining latency	Span share (%)
4	1,203.9	321.3	882.6	26.7
8	1,149.8	320.4	829.4	27.9
16	1,057.7	322.8	734.9	30.5
32	812.8	317.0	495.8	39.0

6.1.2. Recorded-span variation across configured peer counts

Variation of the recorded span across configured peer counts is assessed descriptively. As Section 5.3 confirms directly from the released run manifest, the campaign used 8 live runs per configuration arranged as two complementary 4×4 Latin-square cycles, so that each configuration occupied each within-cycle position exactly twice. Across the four configured peer counts the per-configuration means span -1.05 ms per doubling with a two-point-per-configuration descriptive fit ($R^2 = 0.301$, $p = 0.451$, $n = 4$ configuration means – necessarily low-powered); the extreme configurations (4 and 32 peers, 8 runs each) differ by 4.31 ms with a 90% interval of [4.03, 4.60] ms, entirely inside the ± 10 ms margin (paired $t = 28.77$, $p = < 0.001$; TOST $p < 0.001$). This equivalence analysis is exploratory because the margin was selected after the data were available, rather than specified before execution.

Using the full 32-run joint regression on $\log_2(\text{peers})$ and centred run sequence (Section 5.5) rather than the four configuration means, the small monotonic component in the span becomes statistically detectable ($p = 0.0017$) because of the larger sample, even though its magnitude (-1.05 ms per doubling) remains far inside the equivalence margin; this is a case where higher statistical power changes significance without changing practical relevance, and both results are reported rather than only the one that is easier to state. Because the span does not shrink materially when peers are added, its share of the write path rises, from 26.7% at 4 peers to 39.0% of the smaller total at 32 peers, the opposite of what a designer would assume from the 4-peer figure alone. Splitting the remaining latency into ordering, validation, commit and the peer-count-sensitive part of endorsement needs a transaction timestamped at each Fabric hand-off, which we did not collect (L1).

6.2. Latency by operation class

Policy evaluations (`CheckAccess`, pooled across the non-write operation types recorded) average 280 ms (SD 40) and sensor writes 1,220 ms (SD 60, P95 1,319, P99 1,360), a 940 ms gap between operation classes that differ simultaneously in payload size, world-state access and endorsement work; consensus is common to both, so this is not the cost of consensus. Across 36 role-by-operation cells in the trace, the grant rate ranges from 90.6% to 100.0%; the single cell reaching 100.0% is `Sensor|WriteSensor`, the automated field sensor-write channel rather than a human `CheckAccess` evaluation, and excluding it the remaining 35 `CheckAccess`-relevant cells cluster tightly just above 90.6% (full 36-cell table in `analysis/outputs/02_latency_by_class.json`): a trace in which the recorded operation and role determined the policy path would instead show cells near 0% or 100%, so these labels do not separate policy-distinct executions and do not support an operation- or privilege-specific comparison. The scripted campaign in Section 6.6, where every boundary-crossing request was refused, is the evidence that the policy itself discriminates; whether privilege level or hierarchy depth affects decision cost needs a designed experiment (Section 7).

6.3. Interleaved throughput experiment

Figure 3 reports throughput against offered concurrency for the HRBAC configuration and the access-control-free baseline on identical hardware and workload, recomputed from all 100 live runs.

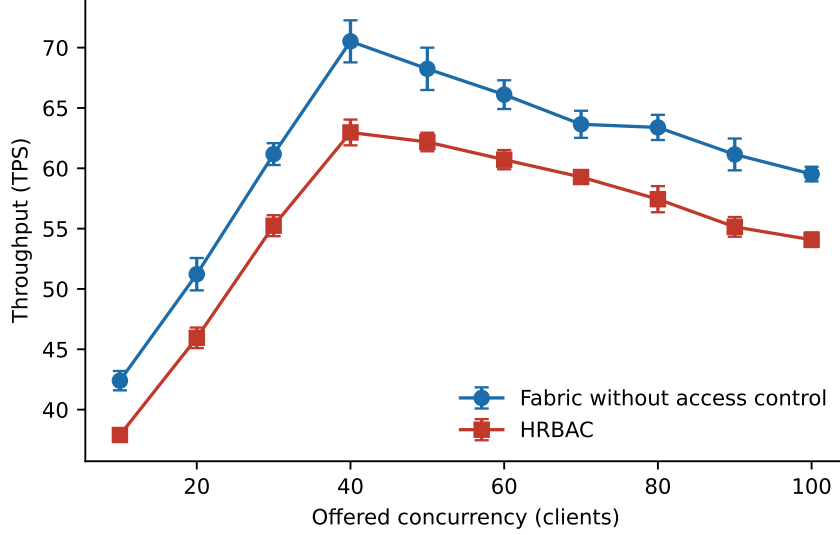


Figure 3: Throughput against offered concurrency for both conditions, with standard deviations over 5 live runs per condition and concurrency cell. The ten concurrency levels were tested in a single fixed ascending sequence within one session; only the two conditions were interleaved within each cell. The highest measured point (40 clients) is the observed peak under this workload; the experiment does not independently identify system capacity or the saturated resource.

The primary analysis is a single factorial regression over every live run, not ten independent per-level Welch tests. Condition, centred concurrency and their interaction are fixed effects, and centred execution position within each concurrency level’s 10-run cell is a covariate:

$$\text{TPS} \sim \text{Condition} \times \text{Concurrency}_c + \text{Position}_{\text{within cell}}. \quad (5)$$

The independent experimental unit is the live run. The campaign used one experimental session rather than multiple sessions or counterbalanced blocks (Section 5.5), so no session or block effect can be estimated and none is claimed. The ten concurrency levels were tested in one fixed ascending order; a run’s raw campaign-wide sequence number is therefore almost collinear with concurrency ($r = 0.995$) and cannot serve as an independent order covariate. Only the two conditions were interleaved within each level.

The condition coefficient estimates the adjusted mean path difference at centred concurrency: the access-control-enabled path was 5.65

TPS lower (95% CI [2.88, 8.41] TPS, $p < 0.001$). The interaction estimates how that difference changes across the tested concurrency levels; its magnitude was -0.0021 TPS per additional client ($p = 0.97$). The within-level position effect was not detected ($p = 0.96$; model $R^2 = 0.317$). Averaged across the ten tested concurrency levels this is a 9.4% reduction (SD 1.2 points, range 6.9–10.7%; full per-level table in `analysis/outputs/04_throughput_per_level.json`). A nonsignificant interaction is not proof that the relative difference is constant outside the tested range: it means no condition-by-concurrency interaction was detected within the precision of this experiment, across ten closed-loop concurrency levels from 10 to 100 clients. All ten per-level comparisons remain significant after Holm–Bonferroni correction (10 of 10).

Across the tested closed-loop concurrency levels, the access-control-enabled path averaged 9.4% lower throughput than the comparison path. No condition-by-concurrency interaction was detected within the precision of this experiment. For the tested hardware, peer configuration, transaction semantics and closed-loop workload, the observed mean difference provides a provisional planning estimate; it should be re-estimated for other deployments and should not be interpreted as a universal authorization derating factor.

Both configurations reached their highest observed throughput at the same tested concurrency, 70.5 TPS for the baseline and 63.0 TPS for HRBAC at 40 clients, after which both decline, HRBAC by 14.1% out to 100 clients. This is the highest measured throughput under the tested closed-loop workload, not an independently established capacity ceiling; concurrency was sampled in steps of ten, so the true peak could sit anywhere in the adjacent interval, and the shared tested peak does not by itself identify which resource saturates (Limitation L3, Section 7.4) – ordering saturation is one candidate, reported for Fabric on x86 by Thakkar et al. [49], but so are CouchDB service times, validation and commit capacity, endorsement queueing and gateway networking, and these clients are also closed-loop, so offered concurrency is a client count rather than a controlled arrival rate. We use *observed peak* throughout and do not use *saturation* unless resource saturation is directly supported by telemetry, which it is not here.

6.4. Logical peer-multiplicity experiment

Endorsing peers are normally added to raise physical capacity. Here we increase configured logical peer multiplicity on four fixed physical hosts and

observe write latency, not capacity, respond. Figure 4 reports write latency as configured peer count increases from 4 to 32 at fixed offered concurrency, across 8 live runs per configuration executed in the counterbalanced order confirmed in Section 5.3.

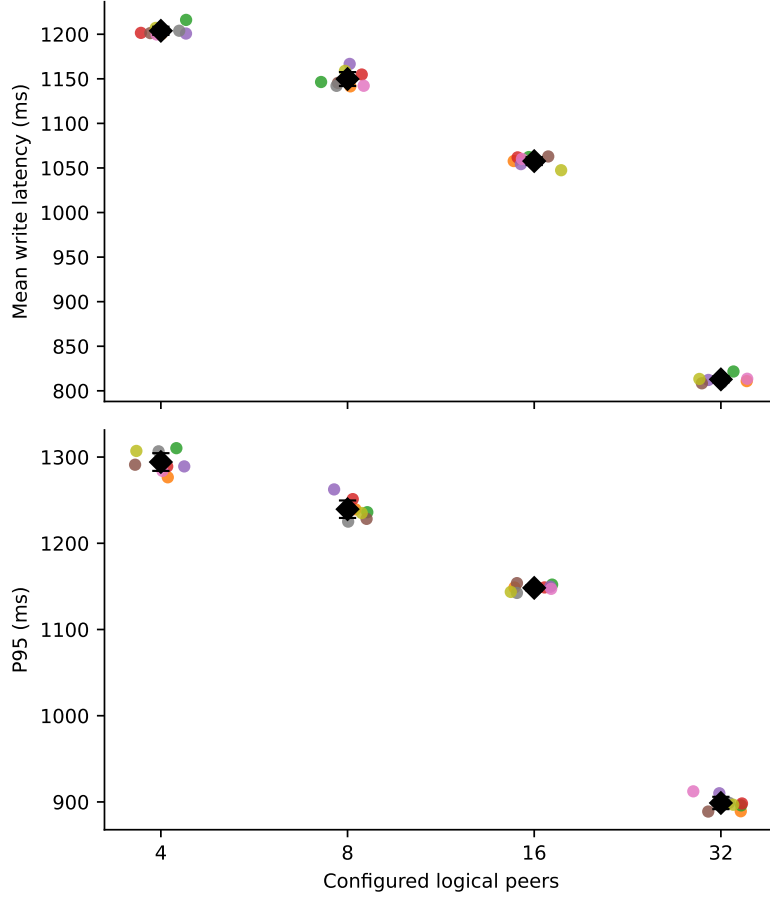


Figure 4: Write latency by configured logical peer multiplicity. Points are all 32 run-level experimental-unit observations (colour identifies the counterbalancing block, i.e. repetition number), jittered horizontally on the $\log_2$ peer-count axis. Black diamonds and error bars show the group mean and 95% confidence interval. The lower panel shows run-level P95 separately. The experiment varied logical peers on four fixed physical hosts and did not measure throughput as a function of peer count.

Mean write latency decreased from 1,203.9 ms to 812.8 ms across the tested configurations, corresponding to a 4-peer versus 32-peer difference of

391.0 ms (95% CI [388.3, 393.8] ms; 8 live runs per configuration, paired within counterbalancing block). P95 decreased from 1,294.2 ms to 898.8 ms, a difference of 395.4 ms. The decline is close to log-linear at -126.5 ms per doubling with $R^2 = 0.891$ across the four configuration means (a two-point-per-configuration descriptive fit, $p = 0.056$); across the four tested configurations, the group means are approximately linear in $\log_2(\text{peer count})$, and this descriptive fit is not presented as a general scaling law. The same effect, estimated jointly with run order on all 32 individual runs (Section 5.5), remains significant at $p < 0.001$ while run order does not ($p = 0.923$) and the counterbalancing block effect is not detected ($p = 0.537$), evidence a single-run, ascending-order design could not have produced. This campaign measured latency only, not throughput, as a function of peer count (Limitation L2, Section 7.4); the separate, interleaved concurrency-throughput comparison of Section 6.3 used a fixed peer configuration and is not informative about peer-count effects.

Every configured peer count above four is hosted on the same four physical machines as the four-peer configuration (`host_id` is constant across all 32 runs), so the comparison varies logical endorser multiplicity rather than genuine physical distribution (L2). Peer queue depth, proposal allocation, CPU utilisation and state-database service time were not recorded, so the experiment does not identify the mechanism. For irrigation control with a two-second responsiveness threshold [50], greater logical peer multiplicity was associated with a larger observed margin relative to that threshold in this configuration. Whether this association persists under other workloads or physical deployments remains untested.

6.5. Revocation-stage observations

Revocation is where short-duration benchmark measurements are least informative, because the mechanism fires rarely and its failure mode is delay rather than error. Over the deployment, 200 lifecycle stage records were logged across 55 days; these are stage observations rather than independent revocations, and Section 6.9.4 explains why end-to-end exposure cannot be recovered from them.

57 of 200 stage records, 28.5%, completed in a delayed state; we give the proportion without a confidence interval, since correlated stages of the same episode are not independent trials. Delayed stages are not slower on the metrics we can observe, since our timers start when publication or gossip begins and any interval spent waiting for a gateway to return is outside

the measurement. Their association with recorded connectivity outages (Section 6.8) motivates a reachability hypothesis, but the trace cannot determine per-episode cause or exclude the propagation protocol as a contributor (Limitation L5, Section 7.4).

The trace contains 973 operational access attempts recorded as denied for revocation, 17.1 per day, spread out rather than concentrated in any single incident, kept separate from the scripted campaign of Section 6.6 since one is field observation and the other synthetic testing. Because the deployment did not collect the denominator of all post-revocation access attempts, this count does not estimate the probability that a revoked credential is successfully blocked; we report it as evidence enforcement fired repeatedly in the field, not as a success rate.

6.6. Authorization-boundary campaign

The security evaluation targets the property the system exists to provide, that no principal acts outside its role, zone or validity window. 8,000 scripted attempts ran, each crossing one specific boundary of the model in Section 3, organised at collection time into 8 named scenarios of 1,000 attempts each. We do not report the results by those scenario names, since repetitions of a scripted request are not independent draws from a population of attacks (no interval is attached to the block rate); the full breakdown groups every attempt by the denial mechanism actually recorded instead.

All 8,000 scripted requests were refused through 6 recorded denial mechanisms in the deployed implementation. We use this phrasing rather than “all attacks were correctly refused” because the campaign does not independently establish consistency with the intended policy, completeness of enforcement, or correctness for untested requests: expected outcomes for the role/operation dimension are checked against an oracle we built and disclose in this version (`analysis/scripts/06_security_oracle.py`), since the `policy-requirements.json` file an earlier draft referenced is not present in this repository (Section 4.5). 2,666 of 8,000 attempts carry a denial reason that makes a role/operation claim (`ROLE_INSUFFICIENT` or `OP_NOT_PERMITTED`); the remainder test zone, certificate, replay or intrusion-detection logic a static role/operation matrix has no opinion on. Of those 2,666, 852 (32.0%) are confirmed against the *deployed* hierarchy of Section 3.2: the deployed `roles.go` agrees the attacking role lacks that permission under our disclosed operation-to-permission mapping. The remaining 1,814 (68.0%)

are not, largely because the deployed hierarchy’s over-broad inheritance (Section 6.9.1) gives many roles a permission our mapping associates with the tested operation even though the request was refused by some other mechanism (zone, certificate, replay); we report this discrepancy rather than resolve it, since the corpus does not carry a separate zone- or resource-ownership attribute that would let us confirm the alternative explanation for each case. This proportion is sensitive to the operation-to-permission mapping, which is our own disclosed construction and not a value taken from the codebase (Table 1 note); it should not be read as a precise false-denial estimate. This extends the independent-oracle check from the role hierarchy alone to the full boundary-attempt corpus’s role/operation dimension; it remains scored against `roles.go` directly for the zone, temporal, replay and identity dimensions the corpus does not separately encode, and the corpus contains no permitted (positive) requests, only denials, so no false-denial rate can be estimated from it either – Limitation L4 is narrowed but not closed. Transport-level attacks, physical tampering and side-channel extraction are not exercised; mutual TLS and constant-time primitives address them architecturally, not empirically.

6.7. *Transmission energy and cryptographic cost*

Reliability of the residue transport is withdrawn (Section 6.9.2); what the campaign measures is cost. Per-event transmission energy falls 48.5% [47.7, 49.3] from 24.13 mJ in single-channel mode to 12.42 mJ with residue encoding ($n = 150$ samples per mode), and the signing pipeline (SHA-256 $15.1\ \mu\text{s}$ + Ed25519 sign $894.8\ \mu\text{s} \approx 910\ \mu\text{s}$; verification $101.9\ \mu\text{s}$) is negligible against the 30-minute duty cycle. This is a single-node bench measurement ($n = 300$ samples per cryptographic operation) and is not generalized across all 50 devices without further instrumentation.

6.8. *Availability*

The online decision path, defined as reachable for all zones, was available 99.36% of the deployment across 3 single-gateway connectivity outages (0.2–8.7 h, 9.3 h total: a LoRa link failure, a power fluctuation and a firmware watchdog reset), with the remaining zones continuing to operate in every case; crediting gateway-hours instead of charging the whole system for one gateway’s outage gives 99.84%. Neither figure is a statement about request success rates, which we did not record per request, nor about data completeness: sensor writes stayed at a constant daily rate including outage days,

since gateways buffer readings while disconnected and replay them on recovery, so a reading accepted after replay was authorized on replay rather than at capture time.

6.9. Implementation-audit and deployment-postmortem results

A structured code-to-specification audit conducted after deployment (Section 5.6) identified three cross-layer defect classes – authorization-policy deviations, invalid residue recovery and signature-verification failure – and one revocation-observability gap that made end-to-end exposure unanswerable. We report them together because the deployed performance measurements and component-level tests did not detect them, and each was found by reading code against prose, and independently reverified for this version directly against the released source files.

6.9.1. Authorization-policy deviations

The role hierarchy in `roles.go` places Certifier on the human inheritance chain rather than beside it, so Agronomist and Farmer receive `ReadAudit` through inheritance, and Certifier itself holds `ReadZone`, so an auditor could read zone sensor data directly – the separation the introduction motivates was never in force for any of the 61 days. An independent second defect is that Agronomist also holds `ControlZone` directly, inconsistent with the “no actuation rights” description in Section 1. Table 1 lists both, together with the specified correction (Section 4.5: specified and logically checked, not implemented as running chaincode in this repository).

6.9.2. Residue-recovery bound violation

As Section 4.2 sets out, recovery from an arbitrary pair of residues is unique only below the smallest pairwise product of the moduli, 9,797. The trace does not record which residue was missing when only two arrived, so a per-pair count cannot be reported. We reconstructed the packed value for all 146,400 committed readings from the released engineering-unit columns using the exact encoding in `esp32/main/main.c` (`encode_sensor_value`: $\text{soil count} \times 201 + \text{temperature bucket}$). The released repository contains no calibration constant converting the released soil-moisture percentage back to the firmware’s raw 12-bit ADC count, so we report the reconstruction under an explicit linear assumption and its reversed-polarity counterpart; under both, 100.0000% of 146,400 readings exceed the smallest recovery bound – an exhaustive count under either assumption, not an inference

from the format’s theoretical ceiling, though the exact packed-value minimum and maximum are calibration-dependent and reported as such in `analysis/outputs/05_crt_quantization.json`.

131,795 readings (90.0%) arrived with two of three residues; since every packed value here exceeded even the largest pairwise bound, each such reconstruction deterministically returned the value reduced modulo its pair’s bound rather than the original, regardless of which residue was missing. All 131,795 two-residue reconstructions are therefore mathematically incorrect, and the original cannot be recovered after the fact, since raw residue packets were not retained.

The specified correction’s $\{253, 254, 255\}$ moduli (bound 64,262) are checked by exhaustive enumeration: every legal input to the specified 8-bit soil encoding gives a maximum packed value of 51,455, below the bound, so every value the specified encoder could produce would be uniquely recoverable. Quantising soil moisture from twelve to eight bits by bit-truncation (the natural embedded-firmware implementation, evaluated by exhaustive sweep of the 12-bit domain rather than a theoretical half-step bound) introduces a maximum error of 0.37% of full scale (RMS 0.21%). Whether this satisfies a particular agricultural application depends on its measurement-accuracy requirement; we do not have a manufacturer tolerance specification or calibration evidence for the deployed soil sensor to certify it in either direction.

6.9.3. Signature-message mismatch and fail-open verification

The defect above would have been caught downstream had the signature path been correct; it was not. The firmware signs SHA-256 over a packed binary struct holding device identifier, reading identifier, raw soil moisture, temperature and timestamp (`esp32/main/main.c`, `sensor_payload_t`), but the gateway verifies an Ed25519 signature over an ASCII rendering built from the decoded value (`gateway/gateway.py`, function `_signature_payload`, which colon-joins the device identifier, reading identifier, decoded value and zone as text), a different message with a different byte layout and a field (zone) the firmware struct does not carry, so verification could not succeed against a genuine sensor signature. The same routine (`verify_signature`) returns `True` immediately whenever a signature is absent (`if reading.signature is None: return True`), a fail-open default apparently intended to let the gateway run without production credentials. Both properties were confirmed by reading the two files directly, not inferred from the trace. The 100.0% signature-validity figure in the trace cannot mean that sensor signatures were

checked and passed, and we withdraw the integrity claim rather than qualify it.

The historical firmware applies Ed25519 to the 32-byte SHA-256 digest of the packed struct, a nonstandard prehash construction retained in the historical package (Section 4.2). Matching the struct layout is therefore not sufficient by itself: a corrected verifier would need to reconstruct the canonical bytes, compute the digest, and verify the signature against that digest, matching the firmware’s byte layout exactly. Such legacy-compatible corrected verification is specified but, as Section 4.5 states, not implemented in this repository; there is no corrected gateway verifier, no regression test, and no published byte-for-byte test vector to check it against. We do not call this a standards-based migration to Ed25519 or Ed25519ph, and no such migration is claimed or implemented.

6.9.4. Revocation observability gap

The revocation trace records lifecycle stages keyed by `event_id`, which we confirmed maps one-to-one to `cert_user_id` in this trace (`analysis/scripts/07_revocation.py`): of 132 subjects, 0 carry all four stage types and 87 appear with only one. End-to-end exposure, the interval from revocation request to enforcement everywhere, is the quantity an operator needs, and it cannot be reconstructed from stage records alone (L5); a correlation identifier that also disambiguates repeated revocations of the same subject over the deployment window is a small change that would make it measurable.

6.9.5. Consequences for historical data validity

Table 4 states, for each audit finding, which historical results remain descriptive of the deployed implementation, which require reinterpretation, which are not measurable and which must be withdrawn. The classifications apply to the historical implementation only; they do not establish corrected-version performance (which is not evaluated, Section 4.5) or the integrity of historical sensor values.

Table 4: Validity classification of historical evidence following the implementation audit.

Historical evidence	Audit finding	Validity classification	Permitted interpretation
Write-path, peer-multiplicity, throughput, cryptographic-timing and energy traces	Policy, encoding, signature and observability defects affect interpretation; corrected-code performance was not measured and no corrected code exists in this repository.	Valid for the deployed implementation	Descriptions of the historical implementation within the stated measurement boundaries; not evidence of corrected-version performance or historical sensor-value integrity.
Authorization decisions, denial mechanisms (Sec. 6.6)	Unintended audit inheritance and Agronomist activation permission; the disclosed oracle mapping is our own construction, not a released file.	Reinterpreted	A record of what the deployed hierarchy enforced, not independent validation of the intended policy.
Sensor readings recovered from two of three residues (90.0%)	The legal packed-value domain exceeded every two-modulus recovery bound.	Withdrawn	Mathematically incorrect values that are not recoverable after the fact.
Signature-validity field	The gateway verified different bytes from those signed by the firmware and accepted missing signatures.	Withdrawn	Cannot be interpreted as evidence that sensor signatures were checked and passed.
Revocation end-to-end exposure	Lifecycle stages lack a within-subject episode-level correlation beyond a single event per subject in this trace.	Not measurable	Only per-stage durations are usable; episode-level exposure and causal attribution cannot be reconstructed.

7. Discussion

7.1. Interpretation of the performance evidence

Four consequences follow, each contradicting a plausible default assumption. The recorded authorization-associated span accounted for roughly a quarter of the historical write path at 4 peers and varied less across the tested peer configurations than the uninstrumented remainder, rising to 39.0% of the smaller total at 32 peers. Because its historical instrumentation boundaries are undocumented, these shares are descriptive and do not establish a transferable optimisation ceiling (Limitation L1). Because expired-role cleanup mutates state inside `CheckAccess`, read-intent checks are endorsed rather than served as peer queries; a read-only decision path with expiry reclaimed by a background sweep would take reads off the consensus path, at the cost of the durable ordered audit transaction a committed `CheckAccess` produces and needing revocation freshness maintained another way, so we call it a high-value candidate worth that trade-off in most deployments rather than an unconditional recommendation. Peers are associated with lower tail latency here; treat throughput separately. Configured logical peer multiplicity is associated with a close-to-log-linear reduction in write latency (Section 6.4), supported by a joint regression on peer count and run order once the earlier order confound is designed out (L2), while the separate, interleaved concurrency experiment found the same observed throughput peak for both arms without locating the saturating resource (L3): “peers buy latency, not capacity” describes only the latency side, and a deployment weighing more endorsers against more capacity should measure both under its own workload. Budget the complete access-control-enabled path empirically. Across the tested closed-loop concurrency levels, this path averaged 9.4% lower throughput than the comparison path, and no condition-by-concurrency interaction was detected. For the tested hardware, peer configuration, transaction semantics and closed-loop workload, the observed mean difference provides a provisional planning estimate; it should be re-estimated for other deployments and should not be interpreted as a universal authorization derating factor. Treat revocation as an unresolved operational risk. 57 of 200 stage records carry the deployment’s delayed label; their association with recorded connectivity outages motivates a reachability hypothesis, but the trace cannot determine per-episode cause or exclude the propagation protocol as a contributor (L5); event-driven credential status checking [51] and constrained-device status protocols such as TinyOCSP [52] remove

the dependency on a scheduled publication reaching a possibly-disconnected gateway, whichever the cause turns out to be.

Direct numerical comparison against prior systems is limited by methodological heterogeneity: hardware, workload, transaction semantics and what is counted as latency all differ (supplementary material), a point Abang et al. [6] make about the Fabric latency literature generally; the comparison we regard as meaningful is the internal one, the same network, hardware and workload with and without the access-control layer (Section 6.3), since it is reproducible from the released artifact.

For a practitioner deciding whether to adopt an HRBAC layer of this shape on similar edge hardware, three operational recommendations follow directly from what was actually measured, each qualified by the limitation that licenses it. First, budget the authorization-associated span as a roughly fixed cost near 321.3 ms rather than a percentage of the write path, since Section 6.1.2 shows the percentage moves with peer count for reasons unrelated to the access-control logic itself (L1). Second, treat additional logical endorsers as a latency lever, not a throughput lever, and measure both independently before provisioning a deployment on their basis (L2, L3). Third, treat certificate revocation as an operational process requiring its own monitoring rather than a mechanism whose correctness the deployment traces already demonstrate; the observability gap in Section 6.9.4 means an operator cannot currently answer "how long was this credential live after revocation" from the released fields alone (L5). None of these recommendations transfers to the specified-but-unimplemented corrected artifacts (L6); they describe operating the system as it was actually deployed.

7.2. Cross-layer lessons from the deployment postmortem

The three defect classes went undetected because nothing compared the implementation against its description: the policy diagram and the inheritance table agreed with each other and disagreed with the code, the encoder guarded a bound the producer violated, and the verifier and signer used different message formats, while tests for each component in isolation passed. Re-verifying these findings directly against the released source for this version, rather than trusting the earlier draft's prose, changed none of the qualitative conclusions and confirmed each defect's exact location (file and function) in Sections 6.9.1–6.9.4. None of the three defects is sophisticated; their absence of detection allowed a 61-day deployment to operate with an implementation that contradicted its own specification.

The broader lesson is methodological. Performance measurements can remain internally valid while the security semantics of the measured system are wrong. A deployment postmortem should therefore pair trace analysis with code-to-specification review and cross-component tests, and should state explicitly which historical variables remain descriptive, require reinterpretation or must be withdrawn (Table 4). A second, narrower lesson follows from Section 4.5: describing a correction in prose is not the same as releasing it, and a reanalysis that only re-derives numbers from data without re-verifying which artifacts actually exist in the repository can silently inherit a documentation gap as if it were a shipped fix. This version treats that check as part of the audit, not an afterthought.

7.3. Security and validity boundaries

The threat model treats orderers and the certificate authority as non-malicious under crash-fault-tolerant Raft, and physical tampering and side-channel extraction as residual risk, architecturally addressed by mutual TLS and constant-time primitives rather than empirically tested. Within that scope, three things this deployment measured are not the same as validating the intended design. Section 6.6 measures refusals against the deployed policy, which Section 6.9.1 shows is not the intended one, and its role/operation oracle is our own disclosed construction rather than a shipped independent artifact (L4), so it is conformance testing of one implementation under one analyst-supplied mapping, not a security proof, and says nothing about an adversary who deviates from the scripted campaign. The postmortem’s corrections are specifications, not patches to running code: as Section 4.5 states plainly, no corrected chaincode, gateway verifier or firmware exists in this repository, so no corrected-implementation performance, correctness or security claim is made (L6). And overhead throughout is attributed by differencing against a baseline with the access-control chaincode removed, which also removes the audit write and the nonce check, so every reported cost bounds the access-control layer rather than an isolated permission decision. Table 4 states, trace by trace, what remains usable after the audit.

7.4. Limitations and required validation

This study has six principal limitations. Table 5 separates each unresolved measurement boundary from its consequence for interpretation. The evidence needed to reduce each limitation is described alongside it in the supplementary material.

Table 5: Principal limitations and their consequences for interpretation.

Limitation	Unresolved boundary	Consequence for interpretation
L1 – Timer boundaries	The exact source-code boundaries of the historical <code>rbac_overhead_ms</code> timer cannot be reconstructed. The pooled operation and role labels also do not isolate hierarchy depth, privilege level or denial mechanism.	The value is an implementation-defined authorization-associated span, not an isolated policy-evaluation cost. Stage attribution requires timestamps at endorsement, ordering, validation and commit boundaries.
L2 – Peer multiplicity and external validity	The peer experiment varied logical peers on four fixed physical hosts (single <code>host_id</code> for all runs). Counterbalancing controls execution position only within this campaign, and throughput was not measured as a function of peer count.	The experiment does not establish physical scale-out effects or generalisation to other hardware, sites, seasons or workloads, and it cannot support a peer-count throughput claim.
L3 – Throughput campaign order and saturation	The two conditions were interleaved within each concurrency level, but the levels were tested in one fixed ascending sequence within a single session. Concurrency was sampled in ten-client steps, and the saturated resource was not instrumented.	The within-level position term cannot exclude campaign-wide drift confounded with concurrency. The true peak may lie between tested levels, and the experiment does not identify the resource responsible for saturation.

Table 5 continued

Limitation	Unresolved boundary	Consequence for interpretation
L4 Authorization-tains denied requests but corpus coverage	– The historical corpus contains denied requests but no permitted requests. Role–operation outcomes were checked against an oracle we disclose and construct for this version (no <code>policy-requirements.json</code> exists in the repository); zone, temporal, replay and identity dimensions remain scored against <code>roles.go</code> -derived logic only.	The corpus cannot estimate a false-denial rate or establish complete conformance to the intended policy; the confirmed/discrepant split is sensitive to our disclosed operation-to-permission mapping.
L5 Revocation observability	– Re- Revocation records are lifecycle stages linked one-to-one to subjects in this trace, but a subject revoked more than once over the deployment cannot be disambiguated, and no stage-to-stage episode key is carried through the pipeline.	End-to-end exposure and causal attribution cannot be reconstructed from the historical trace.
L6 No corrected implementation in this repository	– No No corrected chaincode, firmware or gateway package exists in the repository inventoried for this version, despite the corrections being fully specified and arithmetically checked (Section 4.5). Historical energy was measured on one device, and the deployment covered one site, one season and a sensor-write-dominated workload.	Historical latency, throughput, cryptographic and energy results describe the deployed implementation and must not be assumed to transfer to any future corrected version, because none has been built, tested or measured. Numerical estimates are deployment-specific rather than universal IoT or Fabric constants.

8. Conclusion

This study reports a 61-day agricultural edge-IoT deployment and separately executed live experiments examining the historical implementation’s performance, all recomputed for this version directly from an analyst-supplied raw-data package rather than transcribed from an earlier draft. An implementation-defined authorization-associated span averaged 321.3 ms within the 1,203.9 ms write path. Across 8 counterbalanced runs per configuration, confirmed against the released run manifest, increasing logical peer multiplicity from 4 to 32 on four fixed hosts was associated with a 386.7 ms reduction in the uninstrumented remainder, while the recorded span differed by only 4.31 ms between the extreme configurations. The experiment measured latency rather than physical scale-out or peer-count throughput and did not identify the internal mechanism responsible for the association.

A separate interleaved experiment found that the complete access-control-enabled path averaged 9.4% lower throughput than the comparison path. No condition-by-concurrency interaction was detected across the tested closed-loop range, but the experiment does not establish a universal or constant authorization penalty. Revocation logs contained 200 stage records, including 57 carrying a delayed label, but the trace links stages only to a single event per subject, preventing reconstruction of end-to-end revocation exposure or causal attribution.

Code-to-specification audit, reverified directly against the released source for this version, identified three cross-layer defect classes: authorization-policy deviations comprising unintended audit inheritance and Agronomist actuation permission; invalid two-residue reconstruction for 131,795 readings; and signature-verification failure comprising a signed-message mismatch and fail-open acceptance of missing signatures. It also identified a revocation-observability gap. Corrections to each defect are specified and arithmetically checked, but – unlike an earlier internal draft’s description – no corrected chaincode, firmware or gateway package exists in the repository analyzed here; transfer to any future corrected implementation is therefore not established, and no corrected-version performance or security claim is made. The released raw measurements, execution schedules, the disclosed security oracle, the audit evidence and the analysis pipeline that recomputes every number in this article allow the findings and their limitations to be independently examined.

CRedit authorship contribution statement

Forcha Glen Beloa: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Writing – original draft. **Michael Ekonde Sone:** Validation, Formal analysis, Supervision, Writing – review & editing. **Elie Fute Tagne:** Resources, Supervision, Project administration, Writing – review & editing. **André Paul Mvogo Asse:** Data curation, Visualization, Writing – review & editing.

Ethics and data protection

The University of Buea Faculty of Engineering and Technology departmental research ethics review issued a non-interventional technical-study determination for the collection of operational, performance and access-control traces (FET-CE-2025-014, 14 July 2025). Participating farm operators and administrators provided written informed consent covering data collection, scientific analysis and public release of pseudonymized traces. Participation was voluntary. The released data contain no names, contact information or free-text fields identifying an individual; the outage log’s technical notes describe equipment and network events only. User and sensor identifiers were replaced at collection, and the correspondence between identifiers and individuals was destroyed before preparation of the public dataset. Precise field coordinates were not collected in the released package; the site is reported only at settlement level in the manuscript text. We independently scanned the analyst-supplied raw-data package for names, email addresses, telephone numbers, precise coordinates and other free-text fields as part of preparing this version (`analysis/DATA_QUALITY_REPORT.md`) and found none.

Funding

This work received mobility support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme. That support funded travel and exchange activity only; it had no role in study design, data collection, analysis, interpretation, manuscript preparation or the decision to submit. No other specific grant was received from funding agencies in the public, commercial or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI in the writing process

During the preparation of this work, the authors used generative-AI assistance for statistical reanalysis scripting, drafting support and English-language editing. All statistical results, figures and tables were produced by the released analysis scripts (`paper_iot/V10/analysis/scripts/`) against the raw data and were reviewed by the authors; the authors take full responsibility for the content of the article, its scientific claims and the correctness of the reanalysis.

Acknowledgements

We thank the participating farm operators for their collaboration and practical insight, and acknowledge support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme.

Data availability

The historical implementation, raw and processed traces, live-run metadata, counterbalancing schedules, the statistical analysis pipeline, the disclosed requirements-derived security-oracle mapping, and this article’s supplementary material are publicly available at [https://github.com/forchag/blockchain-fet/tree/\[ARTIFACT_COMMIT_SHA\]](https://github.com/forchag/blockchain-fet/tree/[ARTIFACT_COMMIT_SHA]). The full commit SHA identifies the exact artifact accompanying this article and will be inserted once that artifact commit is finalized; we do not cite the moving default branch or a tag alone. At the time of this analysis the repository’s git history contains a single commit and carries no `v09-deployed-historical` or `v09-corrected` tag; a corrected chaincode, firmware or gateway package is not present as a separate release (Section 4.5). The corrected policy, residue-encoding and signature-verification behaviour described in Section 6.9 is specified and arithmetically checked in this article and its analysis scripts, but has not been implemented, tagged or released, and was not used to generate any measurement reported here. A Zenodo DOI may be minted after the GitHub artifact commit is finalized and is not requested at this stage.

References

- [1] N. Zhang, M. Wang, N. Wang, Precision agriculture: A worldwide overview, *Computers and Electronics in Agriculture* 36 (2002) 113–132.
- [2] S. Benedict, Decentralized access control architecture for iot-enabled smart environments, *Internet of Things* 14 (2021) 100379.
- [3] M. Ferrag, L. Shu, X. Yang, A. Derhab, L. Maglaras, Security and privacy for green iot-based agriculture: Review, blockchain solutions, and challenges, *IEEE Access* 8 (2020) 32031–32053.
- [4] A. Kamilaris, A. Fonts, F. Prenafeta-Boldú, The rise of blockchain technology in agriculture and food supply chains, *Trends in Food Science & Technology* 91 (2019) 640–652.
- [5] A. Rustemi, F. Dalipi, Synergizing iot, ai, and blockchain for smart agriculture: Challenges, opportunities, and future directions, *Internet of Things* 34 (2025) 101778.
- [6] J. Abang, H. Takruri, R. Al-Zaidi, M. Al-Khalidi, Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an iot perspective, *Internet of Things* 26 (2024) 101217.
- [7] R. Sandhu, E. Coyne, H. Feinstein, C. Youman, Role-based access control models, *Computer* 29 (1996) 38–47.
- [8] D. Ferraiolo, R. Sandhu, S. Gavrila, D. Kuhn, R. Chandramouli, Proposed nist standard for role-based access control, *ACM Transactions on Information and System Security* 4 (2001) 224–274.
- [9] N. Ye, Y. Zhu, R. Wang, R. Malekian, Q. Lin, An efficient authentication and access control scheme for perception layer of internet of things, *Applied Mathematics & Information Sciences* 8 (2014) 1617–1624.
- [10] V. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, K. Scarfone, Guide to attribute based access control (abac) definition and considerations, Tech. Rep. SP 800-162, NIST (2014).
- [11] D. Servos, S. Osborn, Current research and open problems in attribute-based access control, *ACM Computing Surveys* 49 (2017) 65:1–65:45.

- [12] J. Hernández-Ramos, A. Jara, L. Marín, A. Skarmeta, Distributed capability-based access control for the internet of things, *Journal of Internet Services and Information Security* 3 (2013) 1–16.
- [13] A. Alshehri, R. Sandhu, Access control models for cloud-enabled internet of things: A proposed framework, in: *Proc. IEEE CIC*, 2016, pp. 530–538.
- [14] S. Sciancalepore, G. Piro, G. Boggia, G. Bianchi, Oauth-iot: An access control framework for the internet of things based on open standards, in: *Proc. IEEE ISCC*, 2017, pp. 676–681.
- [15] O. Novo, Blockchain meets iot: An architecture for scalable access management in iot, *IEEE Internet of Things Journal* 5 (2018) 1184–1195.
- [16] A. Ouaddah, A. Abou Elkalam, A. Ait Ouahman, Fairaccess: A new blockchain-based access control framework for the internet of things, *Security and Communication Networks* 9 (2016) 5943–5964.
- [17] R. Xu, Y. Chen, E. Blasch, G. Chen, Blendcac: A blockchain-enabled decentralized capability-based access control for iots, in: *Proc. IEEE iThings*, 2018, pp. 1027–1034.
- [18] D. Shih, P. Jheng, D. Yen, C. Huang, A blockchain-based access control model for industrial iot, *Sensors* 22 (2022) 2188.
- [19] H. Pajoooh, M. Hammoudeh, F. Alam, S. Brocklehurst, Hyperledger fabric blockchain for securing the edge iot: Systems architecture, experiments and critical observations, *Electronics* 10 (2021) 1338.
- [20] C. Ke, S. Park, Performance modeling of hyperledger fabric for iot access control applications, *Future Internet* 15 (2023) 115.
- [21] G. Putra, V. Dedeoglu, S. Kanhere, R. Jurdak, Trust management in decentralized iot access control system, in: *Proc. IEEE BCCA*, 2021, pp. 1–8.
- [22] M. Usman, A. Jolfaei, M. Tariq, Blockchain-based access control for industrial iot: Domain policy architecture with smart contracts, *Computer Networks* 243 (2024) 110286.

- [23] V. González, L. Sánchez, J. Lanza, J. Santana, P. Sotres, A. García, On the use of blockchain to enable a highly scalable internet of things data marketplace, *Internet of Things* 22 (2023) 100722.
- [24] M. Idrissi, F. Palmieri, Blockchain-based authentication and authorization for iot systems: An end-to-end latency analysis on constrained hardware, *The Journal of Supercomputing* 80 (2024) 1–32.
- [25] A. Tzenetopoulos, P. Katsaros, I. Mavridis, Hlf-kubed: Hyperledger fabric blockchain on edge clusters for iot identity and access control, *Future Generation Computer Systems* 135 (2022) 271–283.
- [26] S. Zaidi, M. Al-Qurishi, M. Abidin, S. Almakdi, Y. Asiri, Fabrication: A hyperledger fabric framework for scalable iot access control, *IEEE Internet of Things Journal* 10 (2023) 12636–12649.
- [27] E. Bandara, X. Liang, K. De Zoysa, P. Foytik, S. Shetty, Fabric-based access control for agricultural supply-chain traceability: A multi-layer policy model, *IEEE Transactions on Dependable and Secure Computing* 18 (2021) 2089–2101.
- [28] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002.
- [29] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, M. Deardeuff, How amazon web services uses formal methods, *Communications of the ACM* 58 (2015) 66–73.
- [30] F. Tian, A supply chain traceability system for food safety based on haccp, blockchain and internet of things, in: *Proc. ICSSSM*, 2017, pp. 1–6.
- [31] Q. Lin, H. Wang, X. Pei, J. Wang, Food safety traceability system based on blockchain and epcis, *IEEE Access* 7 (2019) 20698–20707.
- [32] S. Safeer, P. Gallo, C. Pulvento, Agri-farming with computer vision, iot and blockchain towards climate smart cultivation, *Internet of Things* 34 (2025) 101749.
- [33] Y. Liu, F. Yu, X. Li, H. Ji, V. Leung, Distributed resource allocation in blockchain-based video streaming systems with mobile edge computing, *IEEE Transactions on Wireless Communications* 18 (2019) 695–708.

- [34] X. Wang, Y. Han, V. Leung, D. Niyato, X. Yan, X. Chen, Convergence of edge computing and deep learning: A comprehensive survey, *IEEE Communications Surveys & Tutorials* 22 (2020) 869–904.
- [35] B. Mohanta, A. Awad, T. Elsaka, H. Kheddar, E. Baraka, Smart-contract-based blockchain-enabled decentralized scheme for improving smart-grid security, *Internet of Things* 34 (2025) 101811.
- [36] World Wide Web Consortium, Decentralized identifiers (dids) v1.0, W3C Recommendation, <https://www.w3.org/TR/did-core/> (2022).
- [37] M. Ferdous, F. Chowdhury, M. Alassafi, In search of self-sovereign identity leveraging blockchain technology, *IEEE Access* 7 (2019) 103059–103079.
- [38] S. Tara, S. Zeadally, D. Sookun, Decentralized identity management for iot devices using blockchain and zero-knowledge proofs, *IEEE Internet of Things Journal* 10 (2023) 3817–3827.
- [39] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, T. Watteyne, Understanding the limits of lorawan, *IEEE Communications Magazine* 55 (2017) 34–40.
- [40] A. Neumann, C. Aichele, M. Lindner, S. Wunderlich, Better approach to mobile ad-hoc networking (b.a.t.m.a.n.), IETF Internet Draft draft-openmesh-b-a-t-m-a-n-00 (2008).
- [41] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. Cocco, J. Yellick, Hyperledger fabric: A distributed operating system for permissioned blockchains, in: *Proc. EuroSys 2018*, 2018, pp. 30:1–30:15.
- [42] D. Ongaro, J. Ousterhout, In search of an understandable consensus algorithm, in: *Proc. USENIX ATC 14*, 2014, pp. 305–319.
- [43] M. Yin, D. Malkhi, M. Reiter, G. Gueta, I. Abraham, Hotstuff: Bft consensus with linearity and responsiveness, in: *Proc. ACM PODC 2019*, 2019, pp. 347–356.

- [44] M. Castro, B. Liskov, Practical byzantine fault tolerance, in: Proc. OSDI 99, 1999, pp. 173–186.
- [45] Hyperledger Foundation, Hyperledger fabric v3.0: What’s new, <https://hyperledger-fabric.readthedocs.io/en/release-3.0/whatsnew.html> (2025).
- [46] R. Sandhu, V. Bhamidipati, Q. Munawer, The arbac97 model for role-based administration of roles, ACM Transactions on Information and System Security 2 (1999) 105–135.
- [47] S. Raza, H. Shafagh, K. Hewage, R. Hummen, T. Voigt, Lithe: Lightweight secure coap for the internet of things, IEEE Sensors Journal 13 (2013) 3711–3720.
- [48] H. Garner, The residue number system, IRE Transactions on Electronic Computers EC-8 (1959) 140–147.
- [49] P. Thakkar, S. Nathan, B. Viswanathan, Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: Proc. IEEE MASCOTS, 2018, pp. 264–276.
- [50] J. Lea-Cox, A. Belayneh, M. van Iersel, E. Lichtenberg, G. Kantor, Low-cost soil moisture sensor networks for precision irrigation: Advances and limitations, HortTechnology 23 (2013) 792–798.
- [51] World Wide Web Consortium, Bitstring status list v1.0, W3C Candidate Recommendation Snapshot, <https://www.w3.org/TR/vc-bitstring-status-list/> (2024).
- [52] G. Fedrecheski, J. Rabaey, L. Costa, P. Calcina-Ccori, W. Pereira, M. Zuffo, Tinyocsp: Lightweight certificate revocation for constrained iot devices, Internet of Things 22 (2023) 100740.